HAWKI – C4-Architekturdokumentation

HAWKI (Hawk **I**nterface) ist eine hochschulinterne Web-Plattform, die Studierenden und Mitarbeitenden einen datenschutzkonformen Zugang zu generativen KI-Modellen (z. B. ChatGPT) ermöglicht [1]. Die Architektur wird im Folgenden nach dem C4-Modell in vier Ebenen beschrieben. Zusätzlich werden eine beispielhafte Architekturelle Entscheidung (ADR) und relevante Qualitätsanforderungen (arc42) erläutert.

1. Kontext-Ebene

HAWKI fungiert als Mittler zwischen **Nutzenden** (Studierenden, Lehrenden sowie Administratoren) und externen **KI-Services**. Nutzer melden sich mit ihrer Hochschul-ID über einen bestehenden **Auth-Provider** (z. B. Keycloak OIDC oder Shibboleth SSO) am System an, ein separates KI-Konto ist nicht nötig [1]. HAWKI integriert anschließend verschiedene **Large Language Models (LLMs)** – sowohl Cloud-Modelle (z. B. OpenAI GPT-3.5/4, Google **Gemini**/PaLM) als auch selbst gehostete Modelle (z. B. **GWDG**-Services, **Ollama** für lokale LLMs) [2]. Die **Interaktion** erfolgt über eine Chat-Weboberfläche: Nutzer stellen Anfragen (Prompts) an HAWKI, HAWKI leitet diese an das gewählte KI-Modell weiter und streamt die KI-Antwort zurück an den Browser der Nutzer.

[3] In HAWKI 2 können auch **Gruppenchats** stattfinden: Mehrere Hochschulmitglieder tauschen sich in Chat-Räumen aus (ähnlich WhatsApp) und können die generative KI als zusätzlichen Teilnehmer hinzufügen, der angesprochen werden kann und gemeinsame Antworten liefert [3]. Dies fördert die kollaborative Nutzung der KI. Die Plattform bietet darüber hinaus u. a. einen **„Conversation"**-Modus (freier Chat ähnlich ChatGPT), einen **„Virtual Office"**-Modus (fiktive KI-Expert:innen als virtuelle Assistenten) und **Learning Spaces** zum Erlernen effektiver Prompts [4].

Ein zentrales Architekturziel ist die **Datensouveränität** und **DSGVO-Konformität**: HAWKI speichert keine personenidentifizierenden Nutzerdaten und bündelt alle KI-Anfragen einer Hochschule über einen gemeinsamen Account, sodass extern kein Rückschluss auf einzelne Personen möglich ist [5]. Die Hochschulen behalten dadurch die Kontrolle über ihre Daten und die Nutzung der KI [6]. Im folgenden Diagramm sind die wichtigsten externen Akteure und Systeme im Kontext von HAWKI dargestellt:

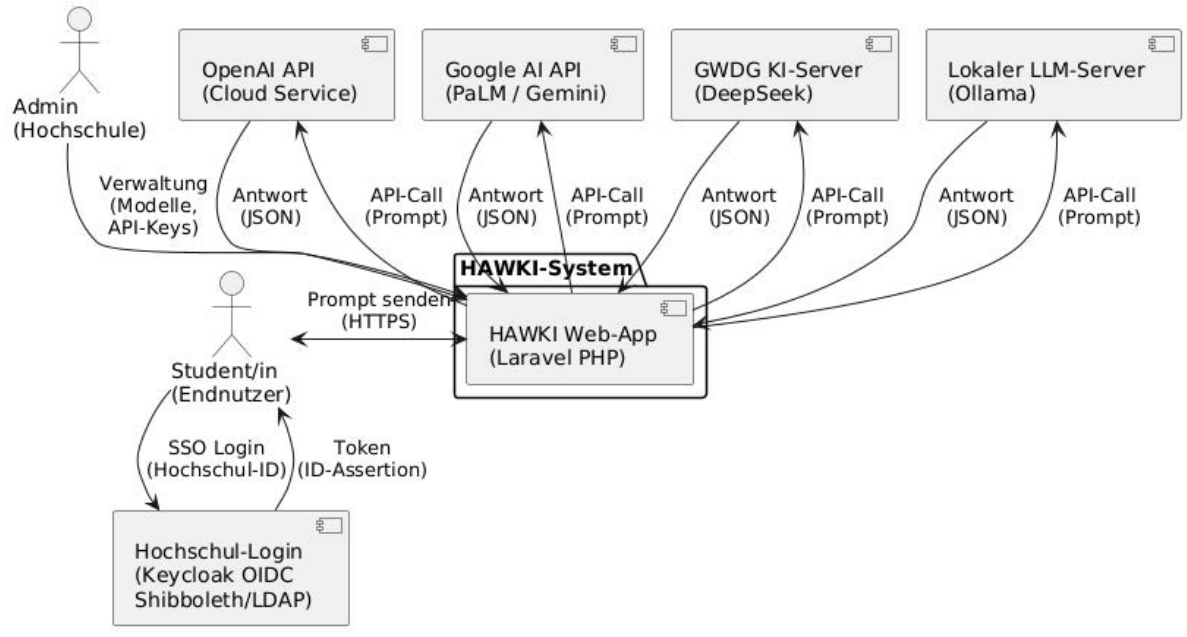


Abb.: **Kontextdiagramm.** Studierende/Lehrende verwenden HAWKI, um über ihre Hochschul-Authentifizierung auf verschiedene KI-Modelle zuzugreifen. HAWKI leitet Anfragen an externe KI-Dienste weiter (OpenAI, Google, GWDG oder lokale Instanzen) und streamt die Antworten zurück an die Nutzer. Administratoren konfigurieren das System (z. B. angebundene Modelle, Ausgabe von API-Schlüsseln für Integrationen). HAWKI selbst stellt sicher, dass keine persönlichen Nutzerdaten an die KI-Provider gelangen [5].

2. Container-Ebene

Die HAWKI-Plattform ist in **Docker-Container** modularisiert und kann per *docker-compose* oder Kubernetes bereitgestellt werden [7]. Die Haupt-Container sind:

- **Webserver (Nginx)** – Frontend-Server, der die Laravel-Webanwendung bedient. Nginx serviert statische Inhalte (HTML/JS/CSS) und leitet API-Anfragen an PHP weiter. In der HAWKI 2-Distribution ist ein vorkonfiguriertes Nginx enthalten (z. B. für SSE-Unterstützung und HTTPS).
- **Application (Laravel PHP)** – Kern der Anwendung (PHP 8 + Laravel Framework). Dieser Container enthält die gesamte Serverlogik: Routen, Controller, Services, KI-Anbindung etc. Er stellt REST-Endpoints und Server-Sent Events bereit. Die Web-App verwaltet Benutzersessions (bei verteilten Instanzen z. B. über Redis) und orchestriert Datenbankzugriffe sowie externe API-Aufrufe.
- **Datenbank (MySQL/MariaDB)** – Persistenzschicht für Chats, Nachrichten, Benutzerdaten (minimal) und ggf. vordefinierte Inhalte. Die DB speichert z. B. Chatverläufe und Prompt-Vorlagen. Standardmäßig wird MySQL verwendet (in der `.env` konfiguriert), alternativ sind andere SQL-Datenbanken via Laravel ORM möglich.
- **Cache/Queue (Redis)** – In-Memory-Datenspeicher, genutzt für Caching von Konfigurationen und ggf. Session-Daten oder Warteschlangen. Redis kann z. B. die Zwischenspeicherung häufig genutzter Prompt-Vorlagen übernehmen. In einer skalierenden Umgebung ermöglicht Redis das Teilen von Sitzungen zwischen App-Instanzen.
- **Optionale Vektor-DB** – Bei Bedarf kann ein Vektor-Datenbank-Container (z. B. **Qdrant**, **FAISS** oder ähnliches) hinzugefügt werden, um Einbettungen (Embeddings) von Texten abzulegen. Dies wäre etwa für semantische Suche oder die Integration externer Dokumente in Chats relevant. HAWKI selbst bringt diesen Container nicht zwingend mit, bietet aber Schnittstellen, um eine Vektor-DB anzubinden (für Funktionen wie kuratierte Wissensdatenbanken oder PDF-Chat).

Die Container kommunizieren über ein internes Docker-Netzwerk. Der Nginx-Container lauscht auf Port 80/443 und leitet eingehende HTTP-Requests an den Laravel-PHP-Container (FastCGI/HTTP). Die Laravel-App verbindet sich über Umgebungsvariablen mit der DB (Standardport 3306) und Redis (6379). Externe KI-Dienste werden aus dem PHP-Container per ausgehendem HTTPS (REST) angesprochen. Antwortdaten von KI-Modellen werden im Laravel-Container in Echtzeit verarbeitet und als **Server-Sent Events (SSE)** an den Browser gestreamt, wodurch Nutzer die Antwort tokenweise live verfolgen können. Die SSE-Kommunikation erfolgt über einen persistenten HTTP-Stream (`Content-Type: text/event-stream`), den z. B. der **StreamController** bedient (siehe Komponenten). Für jede Chat-Nachricht schickt der Browser zunächst eine POST-Anfrage (Prompt) und öffnet dann einen SSE-Kanal, über den die Laravel-App die Antwort häppchenweise zurücksendet.

Authentifizierung & Tokens: Die Nutzer-Session wird nach erfolgreicher SSO-Anmeldung in HAWKI gehalten (z. B. via OIDC-Token in Session oder Cookie). Bei Verwendung von Keycloak/OIDC wird der ID-Token vom Laravel-Backend validiert und der Nutzer als *User*-Objekt geführt. Bei LDAP/Shibboleth-Modus erfolgt die Authentifizierung serverseitig (LDAP-Bind oder Webserver-Auth). HAWKI selbst speichert nur minimale Nutzerinformationen (keine Passwörter – diese verbleiben im Auth-System). Jede Anfrage an externe KI-APIs wird **ohne persönliche Identifikatoren** gestellt – es werden zentrale API-Schlüssel der Hochschule genutzt, sodass extern alle Aufrufe nur der Institution zugeordnet sind, nicht einzelnen Personen [5]. Intern kann HAWKI jedoch Nutzungsstatistiken pro Nutzer sammeln (z. B. Anzahl Anfragen, verbrauchte Tokens), um Missbrauch zu verhindern oder Transparenz zu schaffen. Dieses **Token-Tracking** erfolgt über den *UsageAnalyzerService* und entsprechende Datenbanktabellen (siehe Komponentenebene).

Container-Diagramm:

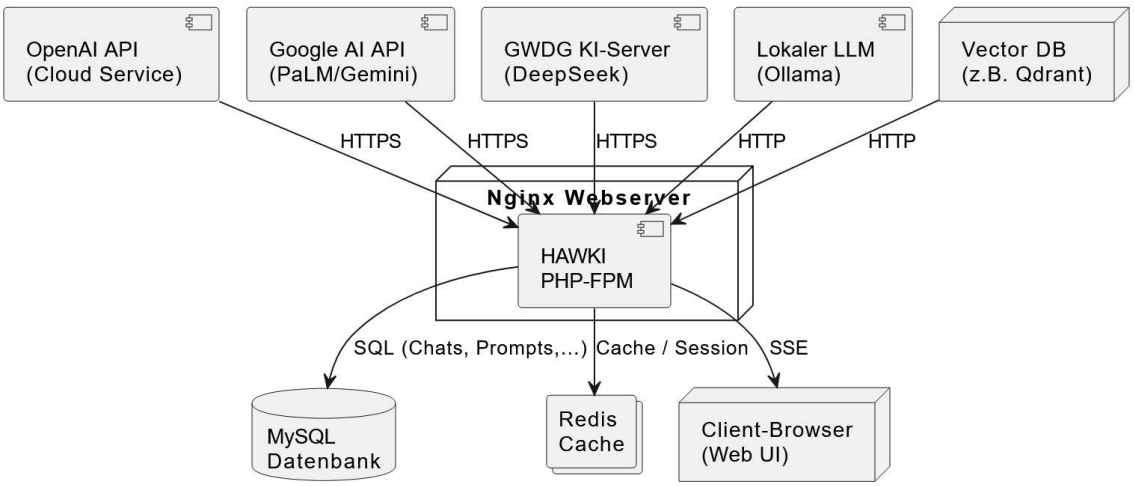


Abb.: **Containerdiagramm.** HAWKI wird in Containern betrieben: Der Nginx-Container empfängt Browser-Anfragen und leitet sie an den PHP/Laravel-Container weiter. Dieser liest/schreibt Daten in die MySQL-Datenbank und nutzt Redis für Cache/Session. Ausgehende Aufrufe zu KI-Provider-APIs erfolgen direkt aus dem PHP-Container. Antworten werden per SSE über Nginx an den Browser gestreamt. Ein optionaler Vektor-DB-Container kann ergänzend verwendet werden, ist aber nicht Teil der Standardinstallation. Die Container-Orchestrierung ermöglicht einfache Deployment- und Skalierungsszenarien (hochschulweit oder lokal für Tests).

3. Component-Ebene

Innerhalb der Laravel-Anwendung lassen sich mehrere **Hauptkomponenten** identifizieren, die gemäß MVC- und Service-Pattern strukturiert sind:

- **Controller:** Die HTTP-Controller koordinieren die Nutzerinteraktionen. Zentral sind der `ChatController` und der `StreamController`.
- `ChatController` verwaltet die Chat-Session und Eingaben. Er bietet z. B. Endpoints zum Starten eines neuen Chats, Senden einer Nutzer-Nachricht (Prompt) und Auslösen der KI-Antwort. Bei einer Anfrage validiert er die Eingabe, legt einen neuen Chat und die Prompt-Nachricht in der Datenbank ab (über Repository/Model) und ruft den KI-Service auf. Auch Funktionen wie Chat-Verlauf laden, Chat beenden oder Export initiieren liegen hier.

- `StreamController` ist für das serverseitige **Streaming der KI-Antwort** zuständig. Er öffnet bei Aufruf eine SSE-Verbindung und schreibt die vom KI-Modell empfangenen Antwort-Teile (Tokens) als Events in den Stream, bis die Antwort komplett ist. So entkoppelt HAWKI die initiale Prompt-Verarbeitung (ChatController) von der schrittweisen Ausgabe (StreamController).

- **Services:** Geschäftslogik und Integrationen sind in Service-Klassen gekapselt. Besonders wichtig:

- `AIConnectionService` (oder ähnlich) übernimmt die **Anbindung an KI-Modelle**. Er fungiert als *Fassade* bzw. Factory, um je nach gewähltem Modell den passenden Provider zu nutzen. Der Service verwendet das **Factory & Strategy Pattern**: Basierend auf einem Parameter (z. B. vom Frontend übermittelter Modell-Typ oder in den Nutzer-Einstellungen hinterlegt) entscheidet er, welche KI-Provider-Implementierung instanziiert wird. Anschließend leitet er die Nutzerfrage an diese Implementierung weiter und erhält die generierte Antwort zurück. So kapselt `AIConnectionService` die Unterschiede der externen APIs und präsentiert dem Controller eine einheitliche Schnittstelle (z. B. eine Methode `generateAnswer(chatId, prompt)` unabhängig vom konkreten Modell).

- `UsageAnalyzerService` überwacht die **Nutzungsauswertung**. Nach jeder KI-Antwort kann dieser Service aufgerufen werden, um z. B. die verbrauchten **Token** zu zählen (viele KI-APIs liefern die Anzahl Tokens der Anfrage/Antwort mit) und in einer Usage-Tabelle zu speichern. Darüber hinaus kann er Summen pro Nutzer/Zeitraum berechnen, um Nutzungsstatistiken bereitzustellen oder Limits durchzusetzen. Auch die Aufbereitung von Chatprotokollen für den Export (inkl. automatischer **Zusammenfassungen**) lässt sich hier verorten – HAWKI 2 bietet eine Exportfunktion, die Konversationen mitsamt einer KI-generierten Zusammenfassung bereitstellt [8]. Dies dürfte durch einen Service (evtl. Teil des UsageAnalyzers oder ein separater *ExportService*) umgesetzt sein, der den Chat-Verlauf nimmt und eine Zusammenfassung via KI erzeugt.

- **Repository/Model:** HAWKI nutzt Eloquent-Modelle (Active Record) für den Datenbankzugriff, teilweise gekapselt durch Repository-Klassen. Wichtige Domänenmodelle sind:

- `User` – repräsentiert einen Benutzer. Da HAWKI keine eigene Benutzerregistrierung hat, werden User-Modelle entweder *on-the-fly* aus den SSO-Infos erzeugt oder minimal gehalten (z. B. nur Matrikelnummer oder Nutzername). Das User-Modell kann Relationen zu Chats besitzen (z. B. Teilnehmer an Chats).
- `Chat` – entspricht einer Chat-Sitzung oder einem Chat-Raum. Enthält Meta-Informationen (Chat-Typ: Single oder Gruppe, Erstellungszeit, Titel) und Relationen zu beteiligten `User`n sowie den zugehörigen `Message`-Objekten. In Gruppenchats können mehrere User mit einem Chat verknüpft sein (N:M Beziehung).
- `Message` (oder `Prompt`) – speichert einzelne Nachrichten innerhalb eines Chats: Nutzer-Eingaben (Prompts) und KI-Antworten als separate Einträge, jeweils mit Referenz auf den Verfasser (User oder ein virtueller „KI-User“) und den Chat. Dies ermöglicht die lückenlose Dokumentation des Gesprächsverlaufs. Alternativ könnte HAWKI die letzte Nutzerfrage und KI-Antwort pro Chat auch überschreiben (falls es einfache single-turn Chats wären), aber da Gruppenchats und Verlaufs-Export unterstützt werden, ist die Speicherung jeder Nachricht wahrscheinlich.

- `PromptTemplate` – (optional) Modell für vordefinierte Prompt-Vorlagen bzw. „virtuelle Assistenten“. Für den **Virtual Office**-Bereich gibt es fiktive Experten als Gesprächspartner; deren Persona und Start-Prompt sind vermutlich in einer Tabelle gepflegt [9]. Dieses Modell hält z. B. den Namen der Rolle (etwa *„KI-Tutor“*), eine Beschreibung und einen initialen System-Prompt, der dem KI-Modell vorangestellt wird, um die Rolle zu etablieren. Der ChatController lädt beim Start eines virtuellen Assistant-Chats das passende PromptTemplate und sendet es unsichtbar mit, damit das KI-Modell in die gewünschte Rolle schlüpft.

- **Provider-Implementierungen:** Die Anbindung unterschiedlicher KI-Anbieter ist durch **Strategie-Klassen** umgesetzt. Es existiert eine gemeinsame Schnittstelle, z. B. ein Interface `AIProvider` oder eine abstrakte Basisklasse, welche grundlegende Funktionalität definiert (z. B. Methode `sendPrompt(prompt, context)` und ggf. Helper für HTTP-Aufrufe, Fehlerbehandlung). Konkrete Implementierungen davon sind:

- `OpenAIProvider` – ruft die OpenAI-API auf (z. B. via `POST /v1/chat/completions`). Er nutzt den konfigurierten API-Key und Endpoint aus der `.env` (`OPENAI_API_KEY`, `OPENAI_API_URL`) [10]. Optional kann diese Klasse auch Microsofts Azure-OpenAI einbinden, indem der API-URL entsprechend gesetzt wird [11] [12].
- `GoogleProvider` (Gemini/PaLM) – Anbindung an Googles KI (z. B. Vertex AI PaLM API). Falls Google **Gemini** als Dienst verfügbar ist, würde diese Klasse die erforderlichen REST-Aufrufe an Google Cloud durchführen (inkl. Authentifizierung über API-Schlüssel oder OAuth). Konfigurationswerte könnten hier ähnlich gehalten sein (API-URL, Key). *Hinweis:* Im Code könnte dieser Provider auch als `GeminiProvider` bezeichnet sein. Google-Modelle werden in HAWKI 2 unterstützt [13], vermutlich via der PaLM2-API oder Bard-API.
- `GWDGProvider` – GWDG (Gesellschaft für wissenschaftliche Datenverarbeitung) hostet eigene LLMs (z. B. **DeepSeek**, **Meta Llama**). Diese Klasse würde die Schnittstelle zu den von der GWDG bereitgestellten Endpunkten herstellen. Möglicherweise bieten sie OpenAI-kompatible APIs an; falls ja, könnte der GWDG-Provider dem OpenAIProvider sehr ähnlich sein, nur mit anderer URL/Key. In jedem Fall kapselt diese Implementierung die Kommunikation zu GWDG-Servern (z. B. HPC-Cluster APIs).
- `OllamaProvider` – Ansteuerung eines lokalen **Ollama**-Servers, der auf dem Host oder im Netzwerk läuft. Ollama dient dem Hosting von Open-Source-Modellen (z. B. LLaMa-Varianten) und stellt eine API bereit. Der Provider kontaktiert diese (standardmäßig läuft Ollama auf einem lokalen Port) und übergibt die Prompt. Damit können in HAWKI lokal betriebene Modelle nahtlos genutzt werden [14] – ein Gewinn für Datenschutz und unabhängige Installationen.

- *Weitere:* Die Architektur lässt sich analog um weitere Provider erweitern, z. B. **Cohere**, **Anthropic**, **AI21** etc., indem einfach neue Implementierungen hinzugefügt werden. Die **Modularität** von HAWKI 2 ist explizit darauf ausgelegt, neue Modelle schnell integrieren zu können [15].

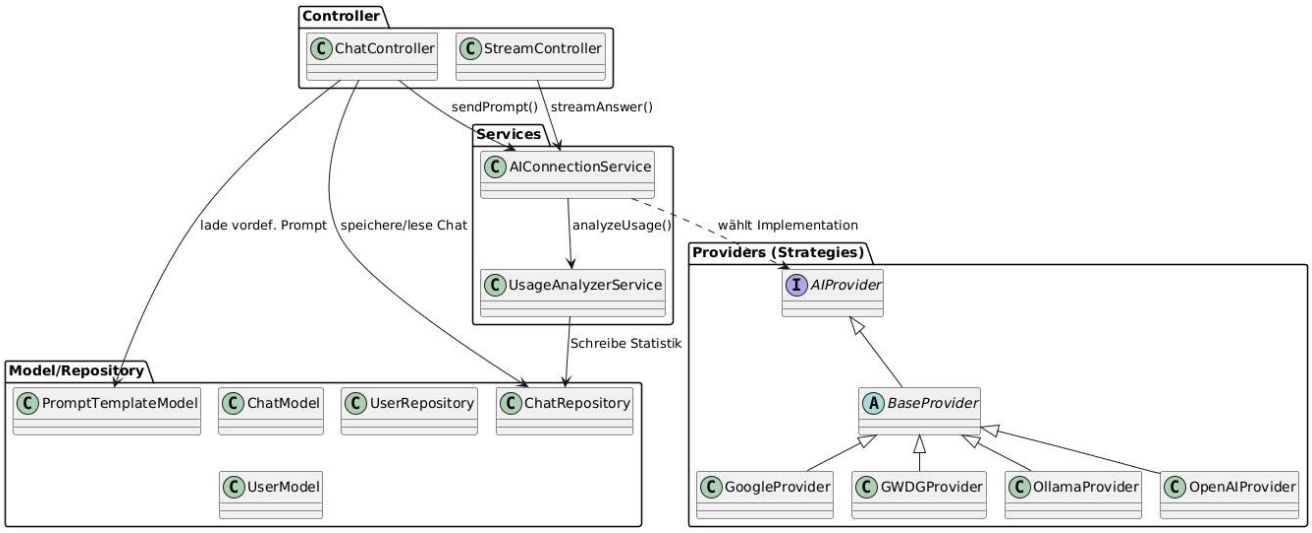


Abb.:=**Komponentendiagramm.** Vereinfachte Sicht auf zentrale Komponenten der HAWKI-Anwendung.= `ChatController` verarbeitet eingehende Chat-Eingaben und nutzt den `AIConnectionService`,=der je nach gewähltem Modell eine entsprechende **Provider-Strategie** (OpenAI, Google, GWDG, Ollama=etc.) zur KI-Berechnung instanziiert. Die KI-Antwort wird dann vom `StreamController` per SSE an=den Nutzer gesendet. `UsageAnalyzerService` protokolliert nach jeder Anfrage die Nutzung (z.B.=Token-Anzahl) in der Datenbank. Datenbankzugriffe laufen gekapselt über Modelle/Repositories (hier= exemplarisch Chat/User). Die Pfeile illustrieren Aufrufbeziehungen und Datenflüsse.

4. Code-Ebene

Auf Code-Ebene spiegelt sich die oben beschriebene Architektur in klar strukturierten Klassen und Entwurfsmustern wider. Hier einige wichtige Aspekte und Patterns im HAWKI-Code:

Kontroller & Routing: In `routes/web.php` sind die Routen definiert, die den `ChatController` und `StreamController` zuordnen. Z. B. könnte es Routen geben wie `POST /chat` (-> ChatController@send) und `GET /chat/stream` (-> StreamController@stream) für SSE. Die Controller-Klassen selbst befinden sich unter `app/Http/Controllers/` und erben von Laravel's Base Controller. Sie nutzen Middleware für Authentifizierung (nur eingeloggte Nutzer) und evtl. Throttling (um Missbrauch zu vermeiden).

Provider-Interface und Basisklasse: Unter `app/Services/AI` (o. ä.) finden sich das Interface `AIProvider` und eine abstrakte Klasse `BaseAIProvider`. Das **Interface** deklariert die notwendigen Methoden, z. B. `generateResponse(Chat $chat, Message $prompt)` und eventuell `supportsStreaming()` etc. Die **Basisklasse** implementiert gemeinsame Funktionalität aller Provider: etwa das Formatieren der Nutzer-Eingaben ins vom API erwartete Format (z. B. Chatverlauf in JSON konvertieren), Aufruf der HTTP-Client-Bibliothek, Error-Handling und ggf. das Zerlegen von Stream-Daten. Einzelne Provider-Klassen (OpenAIProvider, GoogleProvider, etc.) erben von dieser Basisklasse und überschreiben nur, was nötig ist – z. B. Endpunkt-URL, Authentifizierungsmethode oder spezifische Parameter. Dieses Muster erleichtert die Erweiterung: neue Provider können durch Copy-Paste einer bestehenden Implementierung und Anpassung der spezifischen Unterschiede schnell hinzugefügt werden, ohne die gesamte Architektur zu ändern.

Factory-/Strategy-Pattern: Die Auswahl des konkreten Providers erfolgt entweder in `AIConnectionService` selbst oder in einer dedizierten Factory-Klasse. Häufig wird dies simpel gehalten, z. B. via `switch` über einen Provider-Namen: *„if model == 'openai' then provider = new OpenAIProvider();"...*. Alternativ könnte in der Config ein Mapping (`provider_for_model`) gepflegt sein. Unabhängig von der Umsetzung ist dies das **Factory Pattern**, da zur Laufzeit eine von mehreren konkreten Strategie-Klassen instanziiert wird. Das Strategy Pattern liegt darin, dass alle Provider austauschbar das gleiche Interface bedienen – der aufrufende Code (`AIConnectionService`) muss nicht wissen, welcher Provider konkret verwendet wird, er ruft lediglich die Interface-Methoden auf. Dadurch bleibt die Logik sauber getrennt: Der ChatController fragt z. B. `AIConnectionService` nach einer Antwort, aber welche KI (GPT-4, PaLM etc.) diese liefert, entscheidet die Strategie zur Laufzeit. Dieses Design erfüllt das **Open/Closed-Prinzip**: Neue KI-Anbieter erfordern nur das Hinzufügen einer neuen Klasse und einer Factory-Zuweisung, aber keine Änderung am bestehenden Controller-/Service-Code.

Vererbung und Schnittstellen: Neben den KI-Providern nutzt HAWKI eventuell auch Laravel-typische Patterns wie Policy-Classes (für Berechtigungen auf Chats), Events (beim Erzeugen einer Nachricht ein Event „MessageCreated“), die von Listenern (z. B. zum Logging) verarbeitet werden, etc. Auch eigene Exceptions (etwa `OpenAIException`) könnten definiert sein, um Fehler der externen APIs differenziert zu behandeln. Diese tragen zur **Modularität** bei, da Fehlerbehandlung und Speziallogik gekapselt sind. Die Code-Struktur folgt allgemein dem Prinzip der **Single Responsibility** – jede Klasse hat einen klar umrissenen Zweck (Controller = HTTP-Logik, Service = Fachlogik, Provider = Integrationslogik, Model = DB-Mapping).

Modularität & Testbarkeit: Durch die Aufteilung in klar getrennte Layer ist der Code gut testbar. Beispielsweise kann man für einen Komponententest des ChatControllers einen Dummy-`AIConnectionService` injizieren (mittels Laravel's Dependency Injection im Konstruktor), der vordefinierte Antworten gibt – so lässt sich die Controller-Logik unabhängig vom echten KI-Aufruf prüfen. Ebenso können die Provider-Klassen isoliert getestet werden, indem man ihre API-Calls gegen Mock-Endpunkte richtet. Die Konfiguration (in `.env` und `config/*`) erlaubt es zudem, das System flexibel anzupassen, ohne Code ändern zu müssen – z. B. Umschalten von LDAP-Auth zu OIDC oder Wechsel des OpenAI-Modells (durch Ändern der Model-ID in einer Config). Die Verwendung von Env-Variablen für API-Keys und URLs [10] stellt sicher, dass keine Geheimnisse im Code stehen und ermöglicht unterschiedliche Deployments (Test vs. Produktion mit verschiedenen Keys).

Erweiterbarkeit: Das System ist darauf ausgelegt, neue Features einzubinden, wie die Entwickler betonen [15]. Beispielsweise wurde kürzlich die Websuche als zusätzliche Fähigkeit integriert, indem der Google-Provider vor dem Modellaufruf eine Websuche durchführt und deren Resultate in den Prompt einbaut [16]. Auch dies konnte aufgrund der modularen Struktur ohne Einfluss auf andere Komponenten ergänzt werden. Insgesamt zeigt sich auf Code-Ebene ein **sauber gekapseltes**, erweiterbares Design, das schnelle Anpassungen und Innovationen ermöglicht.

Zur Veranschaulichung ein Klassendiagramm-Ausschnitt für die KI-Provider und ihren Gebrauch:

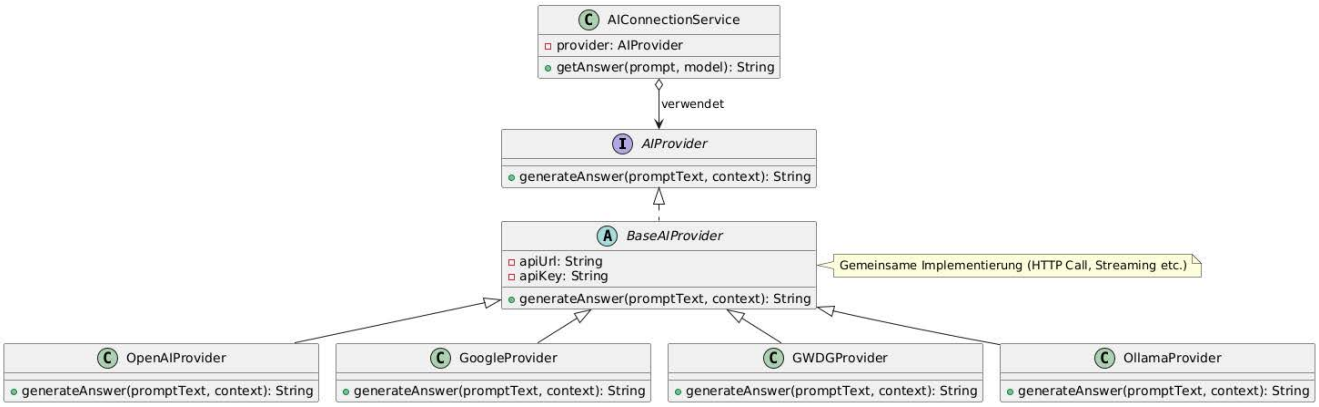


Abb.: **Klassendiagramm (Code-Level).** Das Interface `AIProvider` definiert die Methode zur KI-Abfrage. `BaseAIProvider` implementiert das Interface und bietet Grundfunktionalität für alle Provider. Konkrete Provider-Klassen überschreiben spezifisches Verhalten (z. B. andere API-Parameter). Der `AIConnectionService` hält eine Referenz auf das Interface und erzeugt je nach angeforderter Modellart ein entsprechendes Provider-Objekt (über Factory). Dann nutzt er es, um eine Antwort zu erzeugen. Diese Architektur ermöglicht es, neue Provider-Klassen hinzuzufügen, ohne den Service oder die Controller ändern zu müssen – man erweitert lediglich die Factory-Logik und fügt den API-Schlüssel in der Config hinzu.

ADR-Beispiel: Erweiterung um neuen KI-Provider

Im Folgenden ein **Architectural Decision Record (ADR)** in Kurzform für die Entscheidung, wie neue KI-Modelle in HAWKI integriert werden:

- **Kontext & Problem:** HAWKI unterstützt aktuell mehrere KI-Anbieter (OpenAI, Google etc.). Künftig sollen weitere Modelle (z. B. Anthropic oder lokale LLMs) flexibel eingebunden werden können. Es stellt sich die Frage nach dem geeigneten Erweiterungsmechanismus, um **neue KI-Provider** hinzuzufügen, ohne die bestehende Codebasis zu destabilisieren. Wichtig sind dabei geringe Redundanz, leichte Wartbarkeit und Konfigurierbarkeit der Anbieter-spezifischen Parameter (API-URLs, Keys).

- **Optionen:**

- *Hardcodierung:* Jeden neuen Provider direkt im `ChatController` oder Service mit `if/else` verzweigen. Bei jedem neuen Modell müsste der Code an mehreren Stellen angepasst werden.
- *Strategy Pattern:* Eine allgemeine `AIProvider`-Schnittstelle definieren und für jeden Anbieter eine eigene Implementierung erstellen. Der Controller/Service nutzt polymorph nur die Schnittstelle. Ein Factory-Mechanismus (z. B. ein enum oder Mapping) übernimmt die Auswahl der konkreten Implementierung anhand einer Provider-ID.
- *Externe Plugin-Module:* Die Provider-Logik auslagern, z. B. als separate microservices oder via dynamisches Plugin-System, das zur Laufzeit neue Provider laden kann.

- **Entscheidung:** Gewählt wurde **Option 2: Strategy + Factory Pattern.** Neue KI-Provider werden als separate Klassen implementiert, die ein gemeinsames Interface erfüllen. Die Anwendung enthält eine einfache Factory (z. B. in Form einer Methode im `AIConnectionService` oder

einer zentralen Provider-Registry), welche anhand eines Konfigurationswerts die passende Klasse nutzt.

- **Begründung:** Dieses Muster hält den Code sauber und erweiterbar (**Open/Closed-Prinzip**). Die Logik für jeden Anbieter bleibt gekapselt, was Wartung vereinfacht (z. B. wenn eine API sich ändert, muss nur die entsprechende Klasse angepasst werden). Hardcodierung wurde verworfen, da sie zu Duplication und Fehleranfälligkeit führen würde, wenn mehrere Stellen im Code bei jeder Erweiterung geändert werden müssten. Ein Plugin-System wäre zwar noch flexibler, aber für HAWKI überdimensioniert und würde die Komplexität erhöhen (zusätzliche Schnittstellen, Latenz bei Microservices etc.). Das Strategy-Pattern innerhalb der Laravel-App bietet einen guten Kompromiss aus **Flexibilität** und **Simplicität**.

- **Konsequenzen:** Dank dieser Entscheidung können zukünftige KI-Modelle (z. B. **Cohere** oder **Anthropic Claude**) einfach unterstützt werden, indem die Entwickler eine neue Provider-Klasse schreiben und die Factory-Konfiguration erweitern. Die Testbarkeit bleibt hoch, da jede Provider-Klasse isoliert getestet werden kann. Ein kleiner Mehraufwand besteht darin, die Interface-Verträge stabil zu halten und bei neuen Fähigkeiten (z. B. wenn ein Provider Bildgenerierung statt Text liefern würde) das Interface ggf. zu erweitern. Insgesamt hat sich diese Architektur-Entscheidung bereits bewährt – z. B. konnte die Integration von **Ollama** für lokale Modelle auf diese Weise zügig umgesetzt werden [14].

Qualitätsanforderungen (arc42)

HAWKI wurde gezielt für den Einsatz in Hochschulen entwickelt und erfüllt mehrere wichtige **Qualitätsziele**:

- **Datenschutz (DSGVO-Konformität):** Der Schutz personenbezogener Daten hat oberste Priorität. Es werden *keine* individuellen Nutzerdaten an externe KI-Dienste weitergegeben – alle Anfragen werden im Namen der Hochschule gebündelt gesendet [17]. Nutzer melden sich nur gegenüber der Hochschule an, nicht bei Drittanbietern [1]. Chat-Inhalte verbleiben auf Wunsch vollständig auf hochschuleigenen Servern (durch Nutzung lokaler Modelle wie Ollama). Zudem lassen sich externe Cloud-Modelle durch DSGVO-konforme Alternativen ersetzen (z. B. Einsatz von **Azure OpenAI** anstelle des öffentlichen OpenAI-Services, wodurch Daten in EU-Rechenzentren bleiben und nicht zum Modelltraining verwendet werden [18]). HAWKI speichert Chat-Verläufe und Nutzungsdaten nur so lange wie nötig und bietet Transparenz über die KI-Unterstützung (z. B. Export mit Protokoll [8]). Durch diese Maßnahmen erfüllt HAWKI hochschulische Datenschutzanforderungen und ermöglicht *digital sovereignty* der Institutionen [6].

- **Skalierbarkeit:** Die Architektur ist auf **horizontale Skalierung** ausgelegt. Dank Containerisierung können bei höherer Last mehrere Instanzen der Laravel-App hinter einem Load-Balancer betrieben werden (die Session-Daten liegen in Redis oder es wird stateless JWT verwendet, sodass Instanzen keine Sticky Sessions benötigen). Zusätzliche Nginx-Container können zur Lastverteilung eingesetzt werden. Die Datenbank lässt sich mittels Replikation skalieren (Read-Replikas) und Redis kann in einem Cluster betrieben werden. Zudem können verschiedene KI-Provider parallel genutzt werden – z. B. könnte ein eigener API-Schlüssel pro Fakultät eingerichtet werden, um Last zu verteilen. **Caching** (z. B. von häufig genutzten Antworten oder Embeddings) verbessert die Performance bei vielen Nutzern. Da HAWKI auch als eigenständiges Docker-Image verfügbar ist [7], kann es leicht in Cloud-Umgebungen automatisiert hoch- oder heruntergefahren werden. Für Lastspitzen (z. B. viele gleichzeitige

Prüfungsanfragen) können kurzfristig zusätzliche App-Container gestartet werden. Die nicht-blockierende SSE-Kommunikation sorgt dafür, dass Nutzer auch bei längeren KI-Antworten bedient werden, ohne Thread-Blocking auf Serverseite. Insgesamt ist die Lösung so entworfen, dass sie vom Einzelfall (lokaler Testbetrieb) bis zum hochschuleweiten Dienst mit tausenden gleichzeitigen Nutzern skaliert werden kann.

- **Mehrsprachigkeit & Interkulturalität:** HAWKI legt großen Wert auf **Mehrsprachigkeit** sowohl der Benutzeroberfläche als auch der Modelle. Die Web-Oberfläche ist in *mindestens fünf Sprachen* verfügbar (Deutsch, Englisch, Französisch, Italienisch, Spanisch) [19] – Nutzer können die UI in ihrer bevorzugten Sprache verwenden. Technisch wird dies durch Laravel's Localization erreicht (Übersetzungsdateien für UI-Texte). Darüber hinaus können auch die **KI-Modelle** unterschiedlich sprachliche und kulturelle Hintergründe abdecken. HAWKI unterstützt die Auswahl verschiedener Modelle mit teils unterschiedlichen Trainingsdaten und Sprachen [20]. So können Nutzer z. B. ein deutschsprachig trainiertes Modell (etwa *DeepL.* oder GWDG's *DeepSeek*) auswählen, um eine lokale Perspektive zu erhalten, oder ein Modell wählen, das bestimmte ethische Richtlinien verkörpert. Die Plattform fördert damit *interkulturelle Perspektiven* und erlaubt es, Biases verschiedener Modelle transparent zu machen [21]. Auch die virtuellen Assistenten (Virtual Office) können in verschiedenen Sprachen zur Verfügung stehen, um internationale Studierende zu unterstützen. Insgesamt ist die Software durchgängig Unicode-fähig und unterstützt Multibyte-Zeichen, sodass z. B. auch Eingaben auf Französisch oder Chinesisch verarbeitet werden können – begrenzt nur durch die Fähigkeiten des jeweils gewählten KI-Modells.

Neben diesen Hauptaspekten erfüllen weitere Qualitätsanforderungen die hochschulischen Bedürfnisse: **Wartbarkeit** durch sauberen, modularen Code (siehe oben) ermöglicht schnelle Anpassungen an neue KI-Trends. **Transparenz und Nachvollziehbarkeit** werden durch Logs und die Exportfunktion erreicht [8]. Die **Zuverlässigkeit** wird durch Persistenz von Chats (keine Daten gehen verloren) und Mechanismen wie Modell-Statusanzeigen (Ampelsystem zeigt an, ob ein Modell einsatzbereit ist) erhöht. Insgesamt stellt HAWKI eine flexible, sichere und zukunftsorientierte Architektur bereit, um generative KI im Bildungsbereich effektiv und verantwortungsvoll einzusetzen.

Quellen: HAWKI Quellcode und Dokumentation [1] [2] [17] [19], Entwickler-Berichte [14] [7].

---

[1] [4] [9] HAWKI - didactic interface based on the OpenAI API | Cloudron Forum
https://forum.cloudron.io/topic/12482/hawki-didactic-interface-based-on-the-openai-api

[2] [3] [6] [8] [13] [15] [20] [21] GitHub - hawk-digital-environments/HAWKI
https://github.com/HAWK-Digital-Environments/HAWKI

[5] [17] [18] arxiv.org
https://arxiv.org/pdf/2407.13013

[7] [14] [16] #hawki2 #hawki #interactiondesign | Stefan Wölwer
https://de.linkedin.com/posts/stefanwoelwer_hawki2-hawki-interactiondesign-activity-7315751871174840320-sDwA

[10] [11] [12] HAWKI Update #002 · hawk-digital-environments/HAWKI@146967f · GitHub
https://github.com/HAWK-Digital-Environments/HAWKI/commit/146967f3148e92d1640ffebc21d8914e2d7fb3f1

[19] GitHub - HAWK-Digital-Environments/HAWKI | Stefan Wölwer
https://www.linkedin.com/posts/stefanwoelwer_github-hawk-digital-environmentshawki-activity-7212377305577377792-BDCp